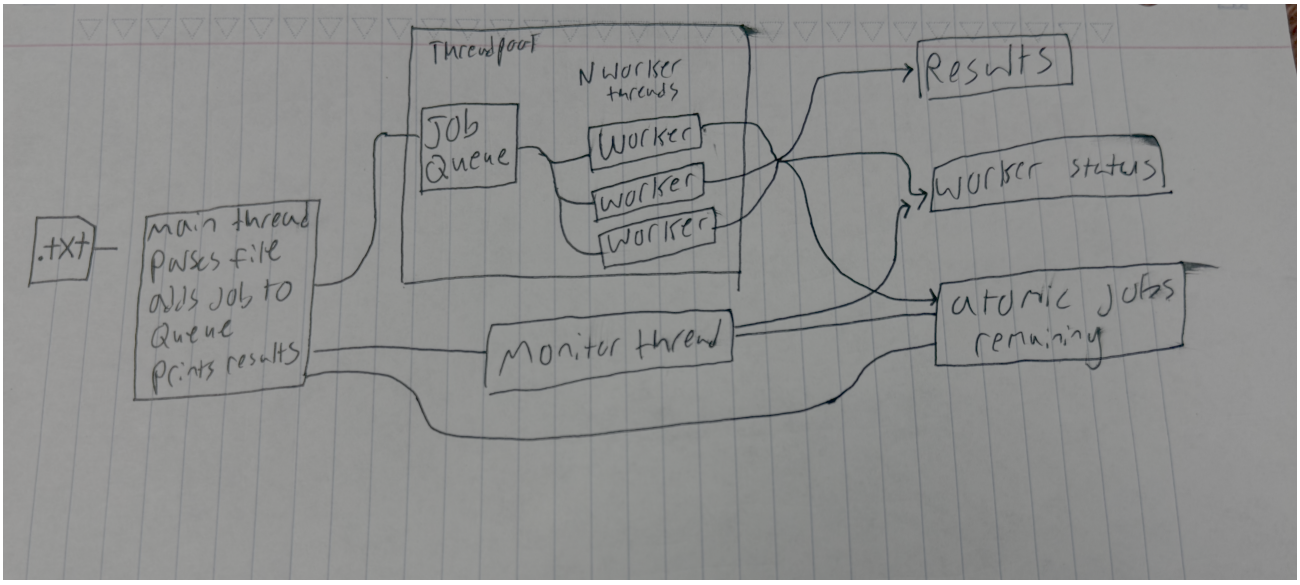
Owen Chipman
Jonah Beck
Dakota Hernandez
Jack Fontenot
Chase Caldwell

Signal Hunters – Submission



Architecture

- **Main thread:** parses the file into `Segment`s, submits one job per segment, calls `shutdown()`, prints results and summary.
- **ThreadPool:** N worker threads, a job queue, a mutex and a condition variable.
- **Monitor:** separate thread that prints worker status about once per second.
- **Job:** analyzes one segment, updates the worker's counters, and appends the result to the results vector.

Job Distribution

Each segment is one job on a single shared queue. Idle workers take the next job from the front, so work is ditributed dynamically.

Shared Data

Data	Protection
`job_queue`, `stopping`	`queue_mutex` + `job_available`
`results` vector	`results_mutex`
`Monitor::stop_requested`	`monitor_mutex` + `stop_signal`
`segments` vector	None needed. Built before workers start, and each job touches only its own element.
`statuses` vector	Each worker writes only its own entry. Monitor reads via atomics.

Atomic Variables

Variable	Why atomic
`segments_processed`, `samples_examined`, `signals_detected`	Written by one worker, read by the monitor and summary while running. Independent counters need no mutex.
`is_busy`	Flag set by the worker, read by the monitor.
`ThreadPool::jobs_pending`	`fetch_add` in `submit`, `fetch_sub` when a job finishes, read by the monitor as "jobs remaining".

Thread-Local Variables

Variable	Why thread-local
`local_worker_id`	The worker's own index, used to tag each result with `worker=N`.
`local_worker_status`	Pointer to this worker's `WorkerStatus`, so a job can update its worker's stats without a lookup or lock.

Sleeping When No Work Is Available

Workers block on `job_available.wait()` until the queue is not empty or `stopping` is set. `submit()` pushes under the mutex and calls `notify_one()`.

Termination

`shutdown()` sets `stopping = true` under the mutex and calls `notify_all()`. Workers drain any remaining jobs, and each exits when the queue is empty and `stopping` is set. `shutdown()` joins all workers. The monitor is stopped through its own codition variable and joined.

Classification

- **STRONG SIGNAL:** longest consecutive run above the threshold is 3 or more.
- **POSSIBLE SIGNAL:** at least one sample above the threshold, but not strong.
- **NO SIGNAL:** no samples above the threshold.

"Above" means greater than the threshold. "Signals detected" counts STRONG and POSSIBLE segments.

Performance Experiment

Input: `signals.txt` (500 segments, 4,165,467 samples). Output was identical for every run on both machines: 500 segments, 4,165,467 samples, 345 signals detected.

Local results (MacBook Pro)

Machine 1: MacBook Pro, Apple M5, 10 cores (4 Super + 6 Efficiency)

Workers	Elapsed time
1	0.002581 s
2	0.001441 s
4	0.001004 s
8	0.000840 s
16	0.000838 s
32	0.001567 s
64	0.002229 s
128	0.002923 s
256	0.004437 s
512	0.007943 s
1024	0.015849 s

Time drops from 0.00258 s at 1 worker to about 0.00084 s at 8 and 16 workers , then flatens. It gets worse from 32 workers on as thread overhead dominates, roughly doubling at each step from 128 to 1024.

Linux server results (earth.ecs.baylor.edu)

Machine 2: Baylor ECS Linux server `earth`, Intel Xeon Gold 6230, 80 logical CPUs

Workers	Elapsed time
1	0.024 s
2	0.014 s
4	0.007 s
8	0.005 s
16	0.004 s
32	0.006 s
64	0.006 s
116	0.009 s (highest count that i could ran)
128	crashed (`std::system_error: Resource temporarily unavailable`)

On Linux the time scales with workers from 1 to 16 (0.024 s to 0.004 s aboout 6x , then gets slightly worse at 32 and 64 as thread overhead outweighs the gains.